```
!pip install "datasets==2.21.0"
```

```
Collecting datasets==2.21.0
  Downloading datasets-2.21.0-py3-none-any.whl.metadata (21 kB)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets==2.21.0) (3.20.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from datasets==2.21.0) (2.0.2)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets==2.21.0) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets==2.21.0) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets==2.21.0) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets==2.21.0) (2.32.4)
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from datasets==2.21.0) (4.67.1)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets==2.21.0) (3.6.0)
Requirement already satisfied: multiprocess in /usr/local/lib/python3.12/dist-packages (from datasets==2.21.0) (0.70.16)
Collecting fsspec<=2024.6.1,>=2023.1.0 (from fsspec[http]<=2024.6.1,>=2023.1.0->datasets==2.21.0)
  Downloading fsspec-2024.6.1-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: aiohttp in /usr/local/lib/python3.12/dist-packages (from datasets==2.21.0) (3.13.2)
Requirement already satisfied: huggingface-hub>=0.21.2 in /usr/local/lib/python3.12/dist-packages (from datasets==2.21.0) (0.36.
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets==2.21.0) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets==2.21.0) (6.0.3)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->datasets==2.21.
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->datasets==2.21.0) (1.4
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->datasets==2.21.0) (25.4.0
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp->datasets==2.21.0) (1.
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp->datasets==2.21.0) (
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->datasets==2.21.0) (0.4
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->datasets==2.21.0) (1.
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.21
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.21.2->da
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datas
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets==2.21.0)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets==2
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets==2
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets==2.21.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets==2.21.0) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets==2.21.0) (2025.2
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->dataset
Downloading datasets-2.21.0-py3-none-any.whl (527 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 527.3/527.3 kB 13.1 MB/s eta 0:00:00
Downloading fsspec-2024.6.1-py3-none-any.whl (177 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 177.6/177.6 kB 19.4 MB/s eta 0:00:00
Installing collected packages: fsspec, datasets
  Attempting uninstall: fsspec
    Found existing installation: fsspec 2025.3.0
    Uninstalling fsspec-2025.3.0:
      Successfully uninstalled fsspec-2025.3.0
  Attempting uninstall: datasets
    Found existing installation: datasets 4.0.0
    Uninstalling datasets-4.0.0:
      Successfully uninstalled datasets-4.0.0
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the
gcsfs 2025.3.0 requires fsspec==2025.3.0, but you have fsspec 2024.6.1 which is incompatible.
Successfully installed datasets-2.21.0 fsspec-2024.6.1
WARNING: The following packages were previously imported in this runtime:
  [datasets,fsspec]
You must restart the runtime in order to use newly installed versions.
```

RESTART SESSION

Task 1: Tải và Tiền xử lý Dữ liệu

```
import torch
from datasets import load_dataset
from collections import Counter

# 1. Tải dữ liệu
print("Đang tải dữ liệu CoNLL-2003...")
dataset = load_dataset("conll2003", trust_remote_code=True)

# 2. Trích xuất câu và nhãn
# Lấy danh sách tên nhãn (ví dụ: 'O', 'B-PER', 'I-PER'...)
label_names = dataset["train"].features["ner_tags"].feature.names

def extract_data(split_name):
    tokens = dataset[split_name]["tokens"]
```

```
    # Chuyển đổi nhãn số sang nhãn string ngay tại đây theo yêu cầu
    tags = [[label_names[tag] for tag in sent_tags] for sent_tags in dataset[split_name]["ner_tags"]]
    return tokens, tags

train_sentences, train_tags = extract_data("train")
val_sentences, val_tags = extract_data("validation")
test_sentences, test_tags = extract_data("test")

# 3. Xây dựng Từ điển (Vocabulary)
word_to_ix = {"<PAD>": 0, "<UNK>": 1}
tag_to_ix = {} # Để padding nhãn, ta sẽ xử lý riêng trong collate_fn hoặc thêm <PAD> vào đây

# Xây dựng word_to_ix từ tập train
for sentence in train_sentences:
    for word in sentence:
        if word not in word_to_ix:
            word_to_ix[word] = len(word_to_ix)

# Xây dựng tag_to_ix từ label_names có sẵn
for name in label_names:
    tag_to_ix[name] = len(tag_to_ix)

print(f"Kích thước bộ từ điển (Vocab size): {len(word_to_ix)}")
print(f"Kích thước bộ nhãn (Tag size): {len(tag_to_ix)}")
print(f"Danh sách nhãn: {tag_to_ix}")
```

```
Đang tải dữ liệu CoNLL-2003...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Downloading builder script: 100%  9.57k/9.57k [00:00<00:00, 43.8kB/s]
Downloading readme: 100%  12.3k/12.3k [00:00<00:00, 33.5kB/s]
Downloading data: 100%  983k/983k [00:00<00:00, 5.34MB/s]
Generating train split: 100%  14041/14041 [00:02<00:00, 6954.99 examples/s]
Generating validation split: 100%  3250/3250 [00:00<00:00, 8380.52 examples/s]
Generating test split: 100%  3453/3453 [00:00<00:00, 8250.13 examples/s]
Kích thước bộ từ điển (Vocab size): 23625
Kích thước bộ nhãn (Tag size): 9
Danh sách nhãn: {'O': 0, 'B-PER': 1, 'I-PER': 2, 'B-ORG': 3, 'I-ORG': 4, 'B-LOC': 5, 'I-LOC': 6, 'B-MISC': 7, 'I-MISC': 8}
```

Task 2: Tạo PyTorch Dataset và DataLoader

```
from torch.utils.data import Dataset, DataLoader
from torch.nn.utils.rnn import pad_sequence

class NERDataset(Dataset):
    def __init__(self, sentences, tags, word_to_ix, tag_to_ix):
        self.sentences = sentences
        self.tags = tags
        self.word_to_ix = word_to_ix
        self.tag_to_ix = tag_to_ix

    def __len__(self):
        return len(self.sentences)

    def __getitem__(self, idx):
        sentence = self.sentences[idx]
        tag_seq = self.tags[idx]

        # Chuyển từ -> index (dùng <UNK> nếu không tìm thấy)
        sentence_indices = [self.word_to_ix.get(word, self.word_to_ix["<UNK>"]) for word in sentence]
        # Chuyển nhãn -> index
        tag_indices = [self.tag_to_ix[tag] for tag in tag_seq]

        return torch.tensor(sentence_indices, dtype=torch.long), torch.tensor(tag_indices, dtype=torch.long)

# Giá trị dùng để padding cho nhãn (để Loss function bỏ qua)
```

```
IGNORE_INDEX = -1

def collate_fn(batch):
    sentences, tags = zip(*batch)

    # Pad câu với giá trị 0 (<PAD>)
    padded_sentences = pad_sequence(sentences, batch_first=True, padding_value=0)

    # Pad nhãn với IGNORE_INDEX (-1)
    padded_tags = pad_sequence(tags, batch_first=True, padding_value=IGNORE_INDEX)

    return padded_sentences, padded_tags

# Tạo DataLoader
BATCH_SIZE = 32
train_dataset = NERDataset(train_sentences, train_tags, word_to_ix, tag_to_ix)
val_dataset = NERDataset(val_sentences, val_tags, word_to_ix, tag_to_ix)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, collate_fn=collate_fn)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False, collate_fn=collate_fn)
```

Task 3: Xây dựng Mô hình RNN

```
import torch.nn as nn

class NERModel(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim):
        super(NERModel, self).__init__()
        # 1. Embedding Layer
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)

        # 2. LSTM Layer (batch_first=True vì input có dạng [Batch, Seq, Feature])
        # Bidirectional=True thường tốt hơn cho NER, nhưng để đơn giản ta dùng 1 chiều trước
        self.lstm = nn.LSTM(embedding_dim, hidden_dim, batch_first=True)

        # 3. Linear Layer
        self.fc = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        # x shape: [batch_size, seq_len]
        embedded = self.embedding(x)  # [batch_size, seq_len, embedding_dim]

        output, (hidden, cell) = self.lstm(embedded) # output: [batch_size, seq_len, hidden_dim]

        predictions = self.fc(output) # [batch_size, seq_len, output_dim]
        return predictions

# Khởi tạo mô hình
EMBEDDING_DIM = 100
HIDDEN_DIM = 256
OUTPUT_DIM = len(tag_to_ix)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = NERModel(len(word_to_ix), EMBEDDING_DIM, HIDDEN_DIM, OUTPUT_DIM).to(device)
print(model)
```

```
NERModel(
  (embedding): Embedding(23625, 100, padding_idx=0)
  (lstm): LSTM(100, 256, batch_first=True)
  (fc): Linear(in_features=256, out_features=9, bias=True)
)
```

Task 4: Huấn luyện Mô hình

```
import torch.optim as optim

# 1. Khởi tạo Optimizer và Loss Function
optimizer = optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss(ignore_index=IGNORE_INDEX)

# 2. Vòng lặp huấn luyện
EPOCHS = 5
```

```
for epoch in range(EPOCHS):
    model.train()
    epoch_loss = 0

    for batch_sentences, batch_tags in train_loader:
        batch_sentences = batch_sentences.to(device)
        batch_tags = batch_tags.to(device)

        # (1) Xóa gradient
        optimizer.zero_grad()

        # (2) Forward pass
        predictions = model(batch_sentences) # [batch, seq_len, num_tags]

        # (3) Tính loss
        # Cần reshape lại để phù hợp với CrossEntropyLoss
        # predictions: [batch * seq_len, num_tags]
        # tags: [batch * seq_len]
        loss = criterion(predictions.view(-1, OUTPUT_DIM), batch_tags.view(-1))

        # (4) Backward pass
        loss.backward()

        # (5) Cập nhật trọng số
        optimizer.step()

        epoch_loss += loss.item()

    print(f"Epoch {epoch+1}/{EPOCHS} | Average Loss: {epoch_loss / len(train_loader):.4f}")
```

```
Epoch 1/5 | Average Loss: 0.0152
Epoch 2/5 | Average Loss: 0.0106
Epoch 3/5 | Average Loss: 0.0084
Epoch 4/5 | Average Loss: 0.0076
Epoch 5/5 | Average Loss: 0.0069
```

Task 5: Đánh giá Mô hình

```
def evaluate(model, iterator):
    model.eval()
    correct_pred = 0
    total_pred = 0

    with torch.no_grad():
        for batch_sentences, batch_tags in iterator:
            batch_sentences = batch_sentences.to(device)
            batch_tags = batch_tags.to(device)

            predictions = model(batch_sentences)
            # Lấy nhãn dự đoán có xác suất cao nhất: [batch, seq_len]
            predicted_tags = torch.argmax(predictions, dim=-1)

            # Tạo mask để loại bỏ các vị trí là padding (IGNORE_INDEX = -1)
            mask = batch_tags != IGNORE_INDEX

            # Chỉ so sánh các vị trí hợp lệ
            correct = (predicted_tags[mask] == batch_tags[mask]).sum().item()
            total = mask.sum().item()

            correct_pred += correct
            total_pred += total

    return correct_pred / total_pred

# Báo cáo kết quả trên tập Validation
val_acc = evaluate(model, val_loader)
print(f"Validation Accuracy: {val_acc:.4f}")

# Hàm dự đoán câu mới
def predict_sentence(sentence):
    model.eval()
    # Tokenize đơn giản bằng split (trong thực tế nên dùng tokenizer chuẩn hơn)
    tokens = sentence.split()
```

```
        # Chuyển thành index
        indices = [word_to_ix.get(word, word_to_ix["<UNK>"]) for word in tokens]
        tensor_input = torch.tensor(indices, dtype=torch.long).unsqueeze(0).to(device) # [1, seq_len]

        with torch.no_grad():
            prediction = model(tensor_input)
            pred_indices = torch.argmax(prediction, dim=-1).squeeze(0).cpu().numpy()

        # Map index ngược lại thành nhãn string
        # Tạo dict ngược: index -> tag
        ix_to_tag = {v: k for k, v in tag_to_ix.items()}

        pred_labels = [ix_to_tag[idx] for idx in pred_indices]

        # In kết quả
        print(f"\nSentence: {sentence}")
        print("-" * 30)
        for word, label in zip(tokens, pred_labels):
            print(f"{word:15} | {label}")

    # Test thử
    predict_sentence("VNU University is located in Hanoi")
```

```
Validation Accuracy: 0.9407

Sentence: VNU University is located in Hanoi
------------------------------
VNU             | B-ORG
University      | I-ORG
is              | O
located         | O
in              | O
Hanoi           | O
```